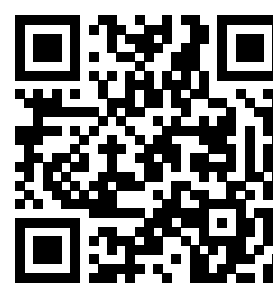


oauth2-passkey

Passwordless Authentication Library for Rust

@ktaka | Tokyo Rust Show & Tell | 2026/03/31



Demo



GitHub



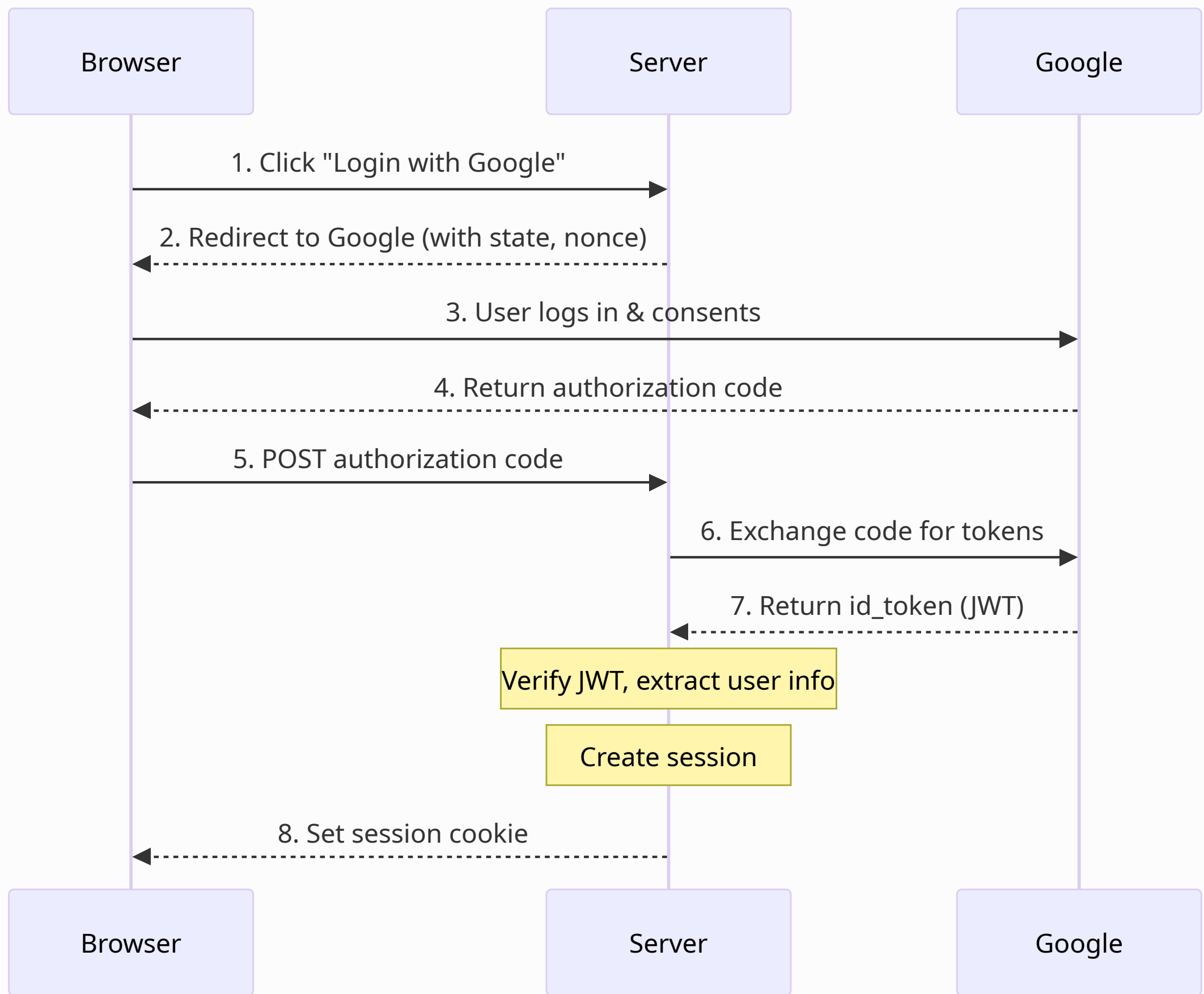
Author

What is oauth2-passkey?

A Rust library that provides **OAuth2/OIDC + Passkey/WebAuthn** authentication for Axum web applications.

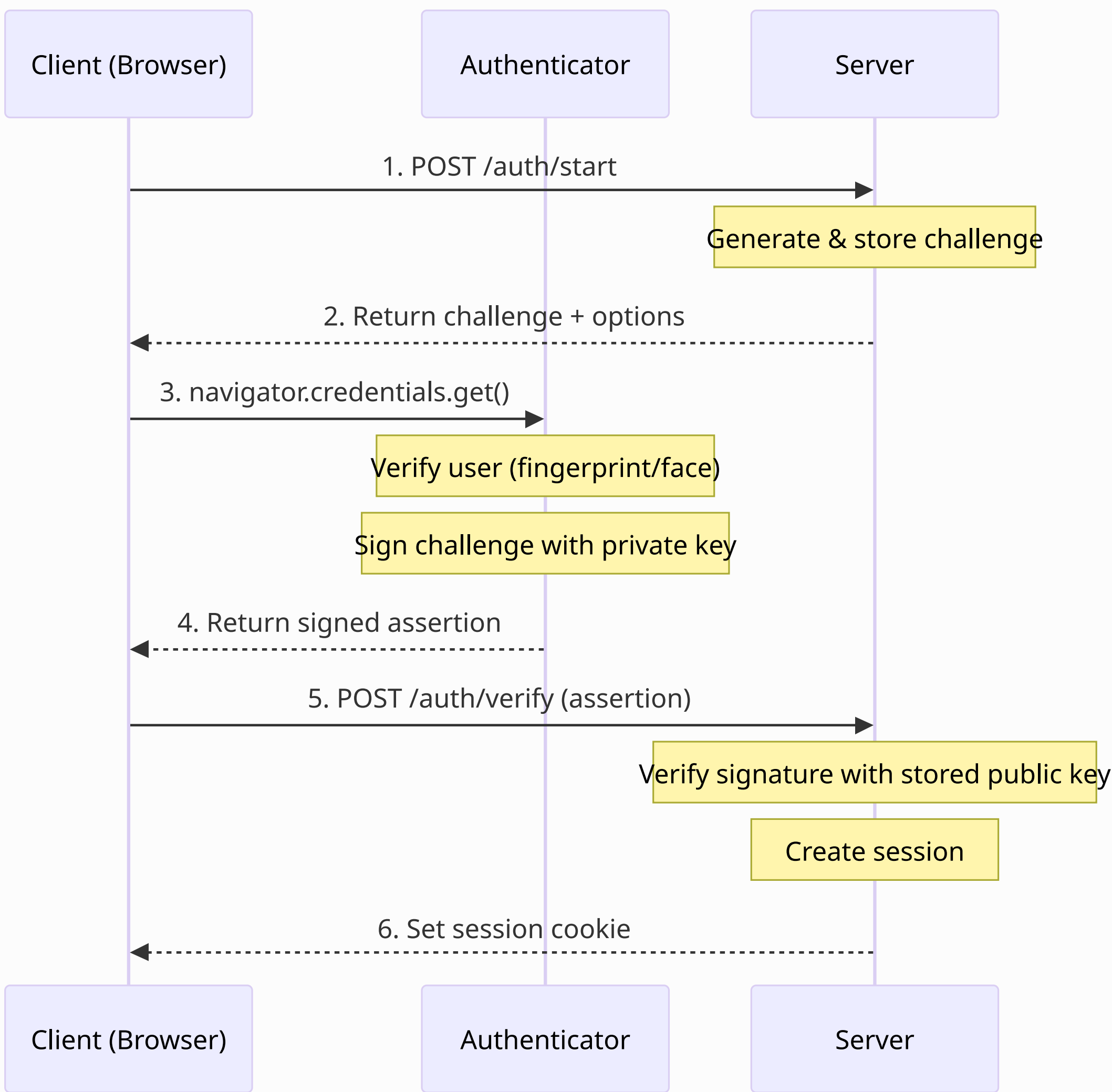
- **Passkey** (WebAuthn) – passwordless, phishing-resistant auth
- **OAuth2/OIDC** – “Sign in with Google” and other providers
- **Account Linking** – multiple auth methods, one user
- **Session management** – cookie-based with CSRF protection
- Published on **crates.io**

How OAuth2/OIDC Works



1. User clicks “Login with Google”
2. Redirect to Google, get authorization code
3. Server exchanges code for **id_token** (JWT)
4. Create session, set cookie

How Passkey/WebAuthn Works



1. Server generates **challenge**
2. Browser calls `navigator.credentials.get()`
3. Authenticator signs challenge with **private key**
4. Server verifies with stored **public key**, set cookie

Authenticators: Google Password Manager, YubiKey, Touch ID, Windows Hello

Multi-DB Support with sqlx

SQLite, PostgreSQL, and MySQL via a single `DataStore` trait:

```
pub(crate) trait DataStore: Send + Sync {
    fn as_sqlite(&self) -> Option<&Pool<Sqlite>>;
    fn as_postgres(&self) -> Option<&Pool<Postgres>>;
    fn as_mysql(&self) -> Option<&Pool<MySQL>>;
}
```

Dispatch via pattern matching – no runtime downcasting:

```
match (store.as_sqlite(), store.as_postgres(), store.as_mysql()) {
    (Some(pool), _, _) => do_sqlite(pool).await?,
    (_, Some(pool), _) => do_postgres(pool).await?,
    (_, _, Some(pool)) => do_mysql(pool).await?,
    _ => return Err(...),
}
```

Usage: Add Auth in 3 Steps

```
use oauth2_passkey_axum::{AuthUser, oauth2_passkey_full_router};

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    dotenv().ok();
    oauth2_passkey_axum::init().await?; // 1. Initialize
    let app = Router::new()
        .route("/", get(index))
        .merge(oauth2_passkey_full_router()); // 2. Merge router
    spawn_http_server(3001, app).await?;
    Ok(())
}

// 3. Protect routes with AuthUser extractor
async fn index(user: AuthUser) -> impl IntoResponse {
    format!("Hello, {}!", user.label)
}
```

Page Protection

AuthUser extractor:

```
async fn protected(user: AuthUser) -> impl IntoResponse { ... }
async fn public(user: Option<AuthUser>) -> impl IntoResponse { ... }
```

Middleware variants:

Middleware	Unauth	User?
<code>is_authenticated_401</code>	401	No
<code>is_authenticated_redirect</code>	Login	No
<code>is_authenticated_user_401</code>	401	Yes
<code>is_authenticated_user_redirect</code>	Login	Yes

Why LazyLock Instead of Axum State?

Typical Axum State:

```
struct AppState { db: PgPool }
let app = Router::new()
    .with_state(state);
// Every handler + 80+ internal
// functions need &AppState
```

oauth2-passkey: LazyLock

```
static GENERIC_DATA_STORE:
    LazyLock<Mutex<Box<dyn DataStore>>>
    = LazyLock::new(|| { ... });
// Any function accesses directly
let store = GENERIC_DATA_STORE
    .lock().await;
```

- **Zero boilerplate** – NO `AppState` for users
- **Env-only config** – `SET DB_TYPE=postgres` IN `.env`
- **Fail-fast init** – `init().await?` catches errors at startup

Integrating with Your App's Database

Link your app data to auth users via `AuthUser.id`:

```
async fn create_todo(
    State(state): State<AppState>,
    user: AuthUser, // from oauth2-passkey
    Form(form): Form<TodoForm>,
) -> Result<Response, ...> {
    db::create_todo(&state.pool, &user.id, &form.title).await?;
    Ok(Redirect::to("/").into_response())
}
```

See `demo-profile` (1:1) and `demo-todo` (1:N) for examples.

Links

- **crates.io**: `oauth2-passkey`, `oauth2-passkey-axum`
- **GitHub**: `github.com/ktaka-ccmp/oauth2-passkey`
- **Demo**: `passkey-demo.ccmp.jp`
- **X**: @ktaka